Agentic RAG Kodlama Lab'ı: Öğrenci Rehberi

Bölümler ve görevler

Bölüm	Görevler
Kurulum	`uv sync`
1. RAG temel yapıları	1 Chunker · 2 Hibrit vector store · 3 Reranker · 4 Input guardrail
2. LangGraph	5 Intent düğümü · 6 Router'lar · 7 Workflow graph
3. Tool ve Agent	8 Arama aracı · 9 Agent kurulumu · 10 Agent düğümü ve agentic graph
Uçtan uca çalıştırma	`docker compose up --build`

Kurulum

1. Repoyu klonla ve kendi branch'ini aç: `git checkout -b ad-soyad`.
2. `uv sync` ile bağımlılıkları kur.
3. `.env.example` dosyasını `.env` olarak kopyala, `GEMINI_API_KEY` değerini gir.
4. `uv run pytest tests/exercises -q` çalıştır. Başlangıçta testler kırmızıdır; bu normaldir.

Görevler boyunca sadece testleri kullanırsın; testler API anahtarı, Qdrant ve Docker olmadan çalışır. Uygulamanın kendisi en sonda Docker ile çalıştırılır (bkz. "Uçtan uca çalıştırma"). Uygulama açılırken chunker'ı ve graph'ı kurduğu için görevler bitmeden başlamaz.

Görevleri bulma ve doğrulama

Yazacağın her yer kodda şöyle işaretlidir:

```
# TODO (Görev 3): Öğrenci Rehberi > Görev 3
# Adım 1: ...
raise NotImplementedError("TODO (Görev 3): Öğrenci Rehberi > Görev 3")
```

`raise` satırını sil, yerine kodunu yaz. Tüm görevleri listelemek için: `git grep -n "TODO (Görev"`.

Tek bir görevi test etmek için: `uv run pytest tests/exercises -v -k gorev3`.

Takılırsan: (1) test mesajını oku, Türkçe ve yönlendiricidir; (2) koddaki `# Adım` yorumlarını ve docstring'i tekrar oku; (3) rehberdeki ipucuna bak; (4) uzun süre ilerleyemezsen öğretmene sor.

Uygulamanın akışı

Bir kullanıcı mesajı şu aşamalardan geçer:

1. **Input guardrail**: Prompt injection varsa reddedilir; kişisel veriler maskelenir.
2. **Intent**: Mesajın niyeti (bilgi sorusu, sohbet, netleştirme, kapsam dışı) ve bağımsız arama sorgusu (`resolved_query`) belirlenir.

3. **Memory recall**: Kullanıcı hakkında daha önce öğrenilen notlar getirilir.
4. **Retrieval + Reranker**: Qdrant'ta hibrit arama yapılır, adaylar LLM ile yeniden puanlanır.
5. **Assistant**: Kaynaklara dayanarak [1], [2] atıflı cevap üretilir.
6. **Judge**: Cevap kaynağa sadık mı, soruyla ilgili mi, uydurma var mı diye denetlenir; kalırsa asistan cevabı yeniden yazar.
7. **Output guardrail**: Cevaptaki kişisel veriler maskelenir.
8. **Memory extract**: Konuşmadan kalıcı notlar çıkarılır.

Belgeler önceden **ingestion** ile işlenir: PDF Markdown'a çevrilir, chunk'lara bölünür, embedding'leri ile Qdrant'a yazılır.

Bu akış iki şekilde kurulur:

- **Workflow graph**: Akış sabittir; bilgi sorularında 4. adım her zaman kodla yapılır.
- **Agentic graph**: 4. adım ayrı düğüm değildir. Asistan bir agent'tır; arama yapıp yapmayacağına kendisi karar verir ve `search_documents` aracını çağırır.

Bölüm 1: RAG Temel Yapıları

Kavramlar

Chunking. Belge tek parça halinde aranamaz; anlamlı küçük parçalara bölünür. Bu projede üç aşama vardır: başlıklara göre (yapıyı korur), anlam kopuşlarına göre (komşu cümlelerin embedding benzerliği düştüğü yerden) ve boyuta göre (çok uzun parçaları keser). Her chunk sayfa, başlık yolu ve deterministik bir kimlik taşır; cevaptaki atıflar buradan gelir.

Embedding. Metni anlamını temsil eden bir vektöre çevirir. Anlamca yakın metinlerin vektörleri de yakındır (kosinüs benzerliği).

Hibrit arama. Dense (embedding) vektör eş anlamlıları yakalar ama tam ifadeleri ("Madde 142") kaçırabilir. Sparse (BM25) vektör anahtar kelimeyi tam eşler ama anlamı bilmez. İki arama ayrı yapılır ve sonuçlar **RRF** (Reciprocal Rank Fusion) ile birleştirilir: her belgenin puanı, listelerdeki sırasının tersinin toplamıdır, `1 / (k + sıra)`.

Reranker. Vektör araması hızlı ama kabadır. Az sayıda adayı (ör. 20) bir LLM'e "bu pasaj soruyu ne kadar cevaplıyor, 0-10" diye puanlatmak en iyi 5'i çok daha isabetli seçer.

Guardrails. Girdide prompt injection engellenir, kişisel veri (TC kimlik, e-posta, IBAN...) `[MASKED_EMAIL]` gibi etiketlerle maskelenir. Çıktıda da aynı maskeleme yapılır.

Görev 1: Chunker splitter'ları

Dosya: `ingestion/document_chunker.py`, `DocumentChunker.__init__`

Üç splitter'ı oluştur:

- `self.header_splitter = MarkdownHeaderTextSplitter(headers_to_split_on=HEADERS, strip_headers=header_splitter_strip_headers)`
- `self.semantic_splitter = SemanticChunker(...)`: `embeddings`, `breakpoint_threshold_type`, `breakpoint_threshold_amount`, `min_chunk_size=min_chunk_chars`, `buffer_size`

- `self.size_splitter = RecursiveCharacterTextSplitter(...)`: `chunk_size=max_chunk_chars`, `chunk_overlap=overlap_chars`, `is_separator_regex=True`, `separators=SENTENCE_SPLIT_RE`, `length_function=len`

İpucu: Parametre adları sınıfların kendi imzasıyla aynıdır; imzadaki değerleri karşılık gelen alana ver.

Görev 2: Hibrit vector store

Dosya: `retrieval/vectordb_service.py`, `get_vector_store`

```
from langchain_qdrant import QdrantVectorStore, RetrievalMode
store = QdrantVectorStore(client=..., collection_name=..., embedding=...,
sparse_embedding=...,
                          retrieval_mode=RetrievalMode.HYBRID,
vector_name="dense", sparse_vector_name="sparse")
```

Servisin `self.client`, `self.embedding` ve `self.sparse_embedding` alanlarını kullan. Koleksiyon oluşturma, yazma ve arama hazırdır; hepsi bu store'u kullanır.

İpucu: `HYBRID` modu dense ve sparse aramayı birlikte yapar ve sonuçları birleştirir. Vektör adları koleksiyon şemasındaki adlarla aynı olmalıdır.

Görev 3: Reranker filtreleme ve sıralama

Dosya: `retrieval/reranking.py`, `rerank_documents`

LLM çağrısı hazırdır; elinde `response.scores` (her biri `RerankItem(id, score)`) vardır. `id`, `candidates` listesindeki indekstir.

1. `response.scores`'u skora göre **azalan** sırala.
2. `score < min_score` olanları, daha önce görülmüş `id`'leri ve `id >= len(candidates)` olanları atla.
3. `candidates[item.id]` belgesinin metadata'sına `"reranker_score"` ekleyerek kopyasını al: `doc.model_copy(update={"metadata": {...}})`.
4. İlk `top_k` belgeyi döndür.

İpucu: LLM aynı id'yi iki kez veya olmayan bir id döndürebilir; çıktısına güvenme. Görülen id'leri bir `set`'te tut. Sıralamayı kesmeden önce yap.

Görev 4: Input guardrail

Dosya: `guardrails/guardrails.py`, `input_guardrail`

1. `self._is_prompt_injection(message)` True ise uyarı logla ve `InputGuardrailResult(is_input_safe=False, message=None)` döndür.
2. Değilse `InputGuardrailResult(is_input_safe=True, message=self._mask_message(message))` döndür. Maskelenecek bir şey yoksa `_mask_message` `None` döner; olduğu gibi bırak.

Sık hatalar

- Reranker'da `score <= min_score` yazmak (eşikteki değer kalmalı) veya sıralamadan önce `top_k` kesmek.
- `doc.metadata["reranker_score"] = ...` ile belgeyi yerinde değiştirmek.
- Injection durumunda maskeli mesaj döndürmek.

Doğrulama

```
uv run pytest tests/exercises/test_01_rag.py -v
```

Bölüm 2: LangGraph

Kavramlar

State. Graph boyunca taşınan sözlük (`models/agent_state_model.py` içindeki `AgentState`). Önemli alanlar: `messages`, `intent`, `resolved_query`, `reranked_documents`, `generation`, `judge_passed`, `retry_count`.

Düğüm (node). State'i okuyup sadece **değişen alanları** içeren bir sözlük döndüren async fonksiyon. LangGraph bu sözlüğü state ile birleştirir. `messages` alanına `{"messages": [AIMessage(...)]}` döndürmek listeye ekler, üzerine yazmaz.

Fabrika kalıbı. Düğüm sadece `state` alır; zincir, servis gibi bağımlılıklar dış fonksiyondan gelir:

```
def make_greeting_node(prefix: str):
    async def greeting_node(state: AgentState) -> dict:
        return {"generation": f"{prefix}, {state.get('user_name', 'misafir')}"}
    return greeting_node
```

Structured output zinciri. Şablon | LLM; LLM cevabı bir Pydantic şemasına zorlanır:

```
chain = TEMPLATE | llm.with_structured_output(Intent)
intent = await chain.ainvoke({"input_text": "...", "history": [...]})   # Intent
örneği
```

Router. State'e bakıp bir sonraki düğümün **adını** döndüren sade fonksiyon. State'i değiştirmez.

Graph.

```
builder = StateGraph(AgentState, context_schema=Context)
builder.add_node("intent", intent_node, retry_policy=RetryPolicy(max_attempts=3))
builder.add_edge(START, "intent")                    # sabit kenar
builder.add_conditional_edges("intent", my_router)   # router'ın döndürdüğü ada
gider
graph = builder.compile(checkpointer=cp, store=store)
```

`RetryPolicy`, geçici bir hatayla (ör. API zaman aşımı) düşen düğümü yeniden dener; LLM çağıran düğümlere verilir.

Görev 5: Intent zinciri ve düğümü

Dosya: `nodes/intent_node.py`

- `build_intent_chain`: `build_chat_model(model_name, temperature=temperature)` ile LLM oluştur, `INTENT_TEMPLATE | llm.with_structured_output(Intent)` döndür.
- `make_intent_node` içinde:
    1. Son kullanıcı mesajı: `get_user_last_message(state)`; geçmiş: `trim_history(state, max_tokens=max_history_tokens)`.
    2. `response = await chain.ainvoke({"input_text": last_user_msg.text, "history": history})`.
    3. `response.confidence < min_confidence` ise niyeti `"needs_clarification"` yap.
    4. `{"intent": ..., "intent_confidence": response.confidence, "resolved_query": response.resolved_query}` döndür.

İpucu: Düşük güvenli bir tahmine güvenmek yerine kullanıcıdan netleştirme istemek daha güvenlidir. Eşit güven kabul edilir (`>=`).

Görev 6: Router'lar

Dosya: `graphs/routers.py`. Tablo yukarıdan aşağı okunur; ilk eşleşen kazanır.

Router	Koşul	Dönüş
`route_after_input_guardrails`	`is_input_safe` yok veya False	`"blocked"`
	aksi halde	`"intent"`
`route_after_memory_recall`	`intent == "knowledge_question"`	`"retrieval"`
	aksi halde	`"assistant"`
`route_after_judge`	`judge_blocked`	`"blocked"`
	`judge_passed`	`"output_guardrails"`
	`retry_count >= max_retries` (yoksa 0)	`"output_guardrails"`
	aksi halde	`"assistant"`
`route_after_output_guardrails`	`len(messages) % batch_size == 0`	`"memory_extract"`
	aksi halde	`"__end__"`

İpucu: Anahtar state'te olmayabilir; `state.get(...)` kullan.

Görev 7: Workflow graph

Dosya: `graphs/graph.py`, `workflow_graph`. İki bölge vardır: (1) düğümleri ekle, (2) kenarlar ve `compile`. Zincirlerin kurulumu hazırdır.

Önce kağıt üzerinde: "Uygulamanın akışı" bölümünü ve router tablosunu kullanarak düğümleri kutu, geçişleri ok olarak çiz. Koşullu okların üzerine koşulu yaz. Sonra kodla.

Düğüm adları (aynen kullan): `input_guardrails`, `intent`, `memory_recall`, `retrieval`, `reranker`, `assistant`, `judge`, `output_guardrails`, `memory_extract`, `blocked`.

Düğüm fonksiyonları: `make_input_guardrails_node(guardrails_service)`, `make_intent_node(intent_chain)`, `make_memory_recall_node()`, `make_retrieval_node(retrieval_service)`, `make_reranker_node(reranker_chain)`, `make_assistant_node(assistant_chain)`, `make_judge_node(judge_chain)`, `make_output_guardrails_node(guardrails_service)`, `make_memory_extract_node(memory_chain)`, `blocked_response_node`.

İpuçları:

- LLM çağıran düğümlere (`intent`, `retrieval`, `reranker`, `assistant`, `judge`) `retry_policy=RetryPolicy(max_attempts=settings.LLM_MAX_ATTEMPTS)` ver.
- Router'ı olan bir düğümden ayrıca `add_edge` yazma.
- `blocked` ve `memory_extract` doğrudan `END`'e gider.
- Son satır: `return builder.compile(checkpointer=checkpointer, store=store)`.

Sık hatalar

- Judge router'ında `judge_passed`'i `judge_blocked`'dan önce kontrol etmek.
- `retry_count > max_retries` yazmak (doğrusu `>=`).
- Aynı düğümden hem `add_edge` hem `add_conditional_edges` ile çıkış tanımlamak.
- Düğüm adında yazım hatası; router'ın döndürdüğü ad ile `add_node` adı aynı olmalı.

Doğrulama

```
uv run pytest tests/exercises/test_02_langgraph.py -v
```

Bölüm 3: Tool ve Agent

Kavramlar

Tool (araç). LLM'in çağırabileceği bir fonksiyon. `@tool` dekoratörü fonksiyonu araca çevirir; **docstring**, LLM'e aracın ne zaman ve nasıl kullanılacağını anlatan açıklamadır. Hazır örnek: `tools/date_tool.py`.

ToolRuntime. Araca, LLM'in görmediği çalışma zamanı bilgisini verir. `runtime.context`, agent çağrılırken verilen `context=` nesnesidir. Bu projede `AgentContext.sources` listesi, bir turda bulunan tüm kaynakları biriktirir.

```
@tool
async def search_documents(query: str, runtime: ToolRuntime[AgentContext]) -> str:
    """Yüklenen dokümanlarda arama yapar."""
    runtime.context.sources.extend(...)
```

Agent. Model ve araç döngüsü: model çağrılır, araç isterse araç çalışır, sonuç modele geri verilir; model cevap verene kadar tekrar eder. `create_agent` bu döngüyü hazır kurar.

Middleware. Döngüye kurallar ekler. `ToolCallLimitMiddleware` bir aracın en fazla kaç kez çağrılacağını sınırlar (sonsuz arama olmaz). `ToolRetryMiddleware` hata veren aracı yeniden dener; yine olmazsa `on_failure` metnini modele verir.

ToolStrategy. Agent'ın son cevabını bir şemaya (`AssistantResponse`) uygun, yapılandırılmış olarak vermesini sağlar.

Görev 8: `search_documents` aracı

Dosya: `tools/retrieve_tool.py`. Boş sorgu kontrolü hazırdır.

Aday getirme ve rerank:

1. `candidate_docs = await retrieval_service.search(query=query, top_k=candidates)`. Burada `top_k` değil, daha geniş aday havuzu olan `candidates` kullanılır.
2. Aday yoksa `NOT_FOUND_ANSWER` döndür.
3. `rerank_documents(chain=reranker_chain, query=query, candidates=candidate_docs, top_k=top_k, min_score=min_score)` çağır. `RerankerError` olursa uyarı logla, `candidate_docs[:top_k]` ile devam et.
4. Belge kalmadıysa `NOT_FOUND_ANSWER` döndür.

Kaynak biriktirme:

1. `start = len(runtime.context.sources) + 1`
2. `runtime.context.sources.extend(documents)`
3. `return f"Bulunan kaynaklar:\n\n{format_sources(documents, start=start)}"`

Neden `start`? Asistan bir turda aracı iki kez çağırabilir. İlk arama [1][2], ikinci arama [3][4] döndürmelidir; aksi halde asistanın "[1]" atfı yanlış belgeyi gösterir.

Görev 9: Agent kurulumu

Dosya: `nodes/assistant_node.py`, `build_assistant_agent`. Değerler docstring'dedir.

```
llm = build_chat_model(model_name, temperature=temperature)
middleware = [
    ToolCallLimitMiddleware(tool_name=SEARCH_TOOL_NAME,
run_limit=settings.MAX_TOOL_CALLS),
    ToolCallLimitMiddleware(tool_name=DATE_TOOL_NAME, run_limit=...),
    ToolRetryMiddleware(max_retries=..., on_failure=_search_tool_failure_message),
]
return create_agent(model=llm, tools=[...], response_format=...,
context_schema=..., middleware=middleware)
```

Araçlar: `make_search_documents_tool(retrieval_service, reranker_chain)` ve `get_today_date`. `response_format=ToolStrategy(AssistantResponse)`, `context_schema=AgentContext`.

Görev 10: Agent düğümü ve agentic graph

Dosya: `nodes/assistant_node.py`, `make_agent_assistant_node`

1. `messages = AGENT_ASSISTANT_TEMPLATE.format_messages(memories=_format_memories(state), intent=state["intent"], resolved_query=state.get("resolved_query") or last_user_msg.text, feedback=format_feedback(state.get("feedback")), history=history)`
2. `agent_context = AgentContext()`. Bu satır `try`'dan **önce** olmalı; hata durumunda da kullanılır.
3. `result = await agent.ainvoke({"messages": messages}, context=agent_context)`
4. En sonda `return _assistant_update(response, agent_context.sources)`

Agent'ta kaynaklar state'ten değil, aracın `agent_context.sources`'a biriktirdiklerinden gelir. Her turda yeni ve boş bir `AgentContext` oluşturulur.

Dosya: `graphs/graph.py`, `agentic_graph`. Workflow graph'tan farkları:

- `retrieval` ve `reranker` düğümleri yoktur.
- Asistan düğümü `make_agent_assistant_node(assistant_agent)` ile kurulur.
- `memory_recall -> assistant` sabit kenardır; `route_after_memory_recall` kullanılmaz.

Sık hatalar

- `start`'ı `extend`'den sonra hesaplamak veya sabit `1` bırakmak.
- `runtime.context.sources = documents` yazmak; önceki kaynaklar silinir.
- Tüm `Exception`'ları yakalamak; beklenen hata sadece `RerankerError`.
- Tek `ToolCallLimitMiddleware` yazmak; her araç için ayrı olmalı.
- `AgentContext()`'i fabrika fonksiyonunda bir kez oluşturmak; kaynaklar turlar arasında birikir.

Doğrulama

```
uv run pytest tests/exercises/test_03_tool_agent.py -v
uv run pytest tests/exercises -q        # hepsi yeşil olmalı
```

Uçtan uca çalıştırma

Tüm testler yeşilse uygulamayı Docker ile başlat:

```
docker compose up --build
```

Bu komut uygulama imajını derler, Qdrant ve uygulamayı birlikte başlatır; ilk derleme birkaç dakika sürer. Tarayıcıda `http://localhost:7860` adresini aç, bir PDF yükle ve iki graph türüyle aynı soruyu sor. Terminaldeki loglarda workflow'da `retrieval` düğümünün, agentic'te ise `search_documents` aracının çağrıldığını gör.

Durdurmak için `Ctrl+C`. Kodu değiştirirsen aynı komutu tekrar çalıştır; `--build` imajı yeni kodla yeniden derler.